Kuku AI Legal & Citizen Assistance

Project by Aditya Jain · B.Tech, IIT Roorkee
A conversational assistant for everyday Indian citizen problems, built with retrieval-augmented generation (RAG).

Why this problem was chosen

People facing unpaid salary, an unreturned rental deposit, a banking dispute or a consumer complaint often need a clear first step. Relevant information is spread across long Acts, government manuals and FAQs. Legal language, unfamiliar procedures and difficulty finding the right document make that first step harder. Kuku was designed to turn a curated collection of these documents into short, understandable guidance and a continuing conversation.

The design priorities were fast, concise replies, English/Hindi/Hinglish interaction, text and voice access, relevant sources, and remembering the facts within an individual chat. Separate saved chats let a user keep different issues apart. Kuku provides legal information and practical assistance; its interface identifies the need for a qualified lawyer when professional advice is required.

How the problem was approached

We chose RAG so answers could draw on documents selected for the project and show source references. The application retrieves relevant passages, supplies them to a pretrained language model, and screens its answer. The project does not train a language model from scratch or fine-tune one. Its main engineering work is dataset preparation, extraction, chunking, search, conversation handling and answer checks.

Topics and practical tasks

Topic group	Intended assistance
Employment and rental issues	Unpaid salary, useful records, employer messages, rental deposits and tenancy questions.
Consumer, banking and cyber issues	Consumer refunds, banking/UPI disputes, ombudsman information and cybercrime reporting guidance.
Citizen support and protection	Free legal aid; family matters; women and child protection; education, disability and senior-citizen rights.

Across these topics, Kuku can explain an available source, suggest ordinary records to retain, draft or revise a message, and ask a focused follow-up. Actual answer coverage depends on the documents retrieved and their applicability. The topic list is the intended scope, rather than a measured guarantee of complete coverage.

Dataset creation & sources

Collection and sorting

The corpus was assembled from official Acts, government publications, citizen guides and FAQ material. PDFs were downloaded and the NALSA FAQ content was collected as text. A sorting script used filenames and extracted content to place documents into Acts or Guides, with uncertain or unreadable files flagged for review. The user reviewed the collection, removed unwanted/unreadable inputs, and kept the final corpus under **data/raw/acts/** and **data/raw/guides/**. The temporary masterdata folder and sorting script were then removed.

Official sources represented in the project

Source / website	Dataset connection
India Code · indiacode.nic.in	Acts and codes, including contract/property law, civil procedure, consumer protection, wage/social-security codes and the newer criminal-law texts.
NALSA · nalsa.gov.in/faqs/	The saved FAQ text explicitly records this source URL. The final extraction contains 86 distinct document/FAQ-number pairs.
NCPCR · ncpcr.gov.in	Child-protection guidance, including the 2024 school bullying and cyberbullying guidelines; its published filename matches the corpus inventory.
Reserve Bank of India · rbi.org.in	Banking grievance guidance. RBIOS01072026.pdf in the inventory matches RBI's published Integrated Ombudsman Scheme 2026 FAQ.
National Cyber Crime Reporting Portal · cybercrime.gov.in	Citizen reporting/user-guide material is represented in the project collection. Exact download URLs for every local PDF were not retained in this summary.

Other listed inputs include Consumer Protection Act FAQs, a cyber-safety guide and FAQPMKISAN.pdf. The download origin and legal currency of each file should be established through its source metadata. This report does not assign an unverified government URL to those individual files.

Extraction and provenance

PyMuPDF extracts physical PDF pages; the TXT parser separates FAQ questions and answers while retaining introductory or malformed text. Conservative Unicode cleaning preserves legal punctuation and numbering. SHA-256 hashes identify duplicate files and stable document IDs. Records carry source paths, page/FAQ locators, titles and optional jurisdiction, effective-date and source-URL metadata. Manifests and checksums link each pipeline stage.

Empty documents and failed pages block normal publication. Explicit partial ingestion is available. Scanned pages and suspicious mixed-script Hindi extraction produce diagnostics; OCR is not implemented. A scanned document removed from the final corpus requires a readable replacement before its content can be searched.

Final ingestion · 58 unique sources · 3,079 records comprising 2,992 PDF-page records, 86 FAQ records and 1 TXT record. There were 67 warnings across 8 sources, 0 errors, 0 empty sources and 0 failed pages. All 58 titles remain manually unverified. The extraction was published as complete under the pipeline's checks.

Models & answer pipeline

Component	Selected technology and purpose
Answer generation	openai/gpt-oss-120b through the Groq API. The project previously experimented with Qwen; the final working configuration uses GPT-OSS.
Speech recognition	whisper-large-v3-turbo through Groq converts microphone or uploaded audio to text.
Embeddings	intfloat/multilingual-e5-small through SentenceTransformers, running locally on CPU; 384 dimensions and a configured 512-token input limit.
Search and reranking	FAISS exact inner-product search over normalized vectors, BM25 keyword retrieval, reciprocal-rank fusion, then cross-encoder/mmarco-mMiniLMv2-L12-H384-v1 reranking.
Application and storage	Python 3.11, Streamlit, SQLite chat history, JSON/JSONL metadata and local FAISS artifacts.

Offline preparation

Extract → chunk → embed → index → validate. Chunking joins consecutive PDF pages while preserving source-page mappings and uses detected section boundaries for Acts. It keeps FAQ context and adds compact document/section headers. The target is 380 tokens with up to 60 tokens of overlap, bounded by the actual embedding tokenizer. E5 inputs use **passage:** for documents and **query:** for searches; prefixes and special tokens are counted.

The final build produced **11,335 chunks and 11,335 FAISS vectors**, each with 384 dimensions. The prior build had 11,493 chunks; revised section detection changed the segmentation. Completed embedding batches are cached for resumable work. Checksums, staging, build locks and replacement recovery reduce inconsistent outputs during ordinary failures; Phase 2 also restores the prior chunks file after a failed build.

Online conversation

Input → conversation context → task routing → retrieval when needed → generation → checks → display. Text is shown before processing. Voice is transcribed and displayed before the same text workflow runs. The assistant uses recent turns, user corrections and records already mentioned to interpret follow-ups. Practical recordkeeping questions can use a checklist path; questions about legal sufficiency, rights or procedures use sources. The current implementation also has a narrow predefined first response for the tested unpaid-salary wording.

Hybrid search takes up to 40 semantic and 40 lexical candidates. Query expansion retains the original terms. Reciprocal-rank fusion uses k = 60; up to 24 candidates are reranked in batches of 8, and up to 6 passages become answer context. These are retrieval limits, not proof of relevance.

Response controls

A fixed system prompt is sent on each generation request. It specifies Kuku’s role, scope, language matching, short conversational replies, source-grounded legal claims and focused follow-ups. The normal target is 4 sentences, with flexibility for requested drafts and checklists. Temperature is 0.1, the completion cap is 1,024 tokens, timeout is 30 seconds, and SDK retries are capped at 2. Mechanical checks screen citation IDs, selected uncited claims, reply language, unwanted draft formats and some deadline mismatches; one answer-revision attempt is allowed.

Project structure & results

Location	Responsibility
app.py · config.py	Streamlit chat/voice interface and saved-chat controls; validated environment settings and project paths.
src/assistant_service.py · models.py	Conversation context, task routing, prompting, answer checks and CaseState serialization.
src/ingestion.py · chunking.py	PDF/TXT/FAQ extraction, provenance, diagnostics and tokenizer-bounded passages.
src/embeddings.py · retriever.py	CPU embeddings, cache and FAISS lifecycle; hybrid ranking, expansion and reranking.
src/groq_client.py · safety.py · utils.py	Groq chat/transcription, bounded API requests, urgent notices and citation helpers; cleaning, hashes and atomic JSON writes.
scripts/01_ingest.py	Build processed/extracted.jsonl, reports and data/manifest.json.
scripts/02_build_index.py	Build processed/chunks.jsonl and verified vector_store artifacts.
scripts/03_check_data.py	Check raw inventory, extraction, chunk provenance, token counts and FAISS alignment.
data/raw/ · processed/ · vector_store/	Source Acts/Guides; extracted/chunked records and reports; index.faiss, metadata and index_manifest.json.
.env · .env.example · .streamlit/config.toml	Private API key and working settings; shareable settings template; Streamlit configuration. requirements.txt lists dependencies; .venv holds the local Python environment.

Interface and operation

The UI supports text, microphone recording and audio upload with a 25 MB application limit. It shows the user's message/transcript during processing and supporting source labels after replies. A sidebar provides separate chats, new-chat and delete controls. SQLite saves conversation state and citations at **~/.kuku/chats.sqlite3**. The current local installation shares this history at the OS-user level; it is not an authenticated multi-user deployment.

From the project root, stop Streamlit before rebuilding. Run **scripts/01_ingest.py**, then **scripts/02_build_index.py --require-complete**, then **scripts/03_check_data.py --require-complete --verify-tokens** with the virtual-environment Python. Continue after each successful phase. Launch the interface with **python -m streamlit run app.py**. Ordinary app restarts reuse the stored index.

What has been demonstrated

The final integrity check passed for all 58 documents and all 3,079 extracted records. It verified 11,335 indexed chunks, including 11,240 PDF chunks with valid page references, aligned FAISS metadata, actual tokenizer counts and 86 distinct FAQ pairs. Live screenshots demonstrated concise first-step guidance, email drafting, practical checklists, bank-statement follow-ups and a switch to cited replies. Local regression checks used simulated model responses to exercise routing, validation/retry behavior, saved-chat isolation and build recovery.

Remaining evaluation. A valid citation marker does not establish that its passage supports the claim. The latest cited salary reply made an unverified claim about bank statements being insufficient by themselves. Full legal accuracy, retrieval relevance and multilingual quality have not been benchmarked; latency has not been systematically measured. Next evaluation should inspect retrieved passages against individual claims, test paraphrases and longer conversations, review source dates/jurisdictions, and resolve extraction warnings.

Reference trail · Project code, supplied inventory and final run logs. Official source entry points: indiacode.nic.in; nalsa.gov.in/faqs/; ncpcr.gov.in; RBI ombudsman FAQ; cybercrime.gov.in. Source entry points and download provenance are distinct.